

SUBSYSTEM 1 · PHASE 1 · COMPLETED

FastAPI

Ingestion Engine

LogiTwin AI Engine — Subsystem 1 Complete Documentation

Every file, every command, every decision — documented for reference.

7	8	3	LIVE
Files Built	API Endpoints	Node Types	API Status

FastAPI 0.115	Pydantic v2	Neo4j 5.20	Python 3.11	uvicorn	Docker	asyncio
---------------	-------------	------------	-------------	---------	--------	---------

01 — Project Structure

Every file has one job. No file does two things. This is the complete directory layout of Subsystem 1.

```
logitwin/
■■■■ .env # All secrets & config – never committed to Git
■■■■ requirements.txt # Pinned dependencies for the full 4-phase build
■■■■ docker-compose.yml # Orchestration file (Phase 1+)
■■■■ Dockerfile # Container definition for FastAPI
■■■■ app/
■■■■ __init__.py
■■■■ config.py # FILE 1: Pydantic BaseSettings – reads .env
■■■■ models.py # FILE 2: All Pydantic v2 data models
■■■■ db.py # FILE 3: Async Neo4j driver pool
■■■■ main.py # FILE 6: FastAPI app – wires everything
■■■■ routes/
■ ■■■■ __init__.py
■ ■■■■ ingest.py # FILE 5: All POST/GET ingest endpoints
■■■■ services/
■■■■ __init__.py
■■■■ graph.py # FILE 4: All Cypher MERGE queries
```

Setup Commands — Run Once

```
cd ~/projects/logitwin
mkdir -p app/routes app/services
touch app/__init__.py app/routes/__init__.py app/services/__init__.py
touch app/config.py app/models.py app/db.py app/main.py
touch app/routes/ingest.py app/services/graph.py
touch .env requirements.txt docker-compose.yml Dockerfile
pip install fastapi uvicorn[standard] pydantic>=2.0 pydantic-settings neo4j httpx python-dotenv
structlog celery redis pytest pytest-asyncio
```

02 — All 7 Files — Complete Code Reference

File 1

app/config.py

Pydantic BaseSettings — reads all .env variables

Purpose

- > Single source of truth for all configuration values
- > lru_cache ensures .env is read exactly once for the entire app lifetime
- > Every other file imports get_settings() — nothing else reads .env directly

Key Design Decisions

- > NEO4J_PASSWORD has no default — app crashes on startup if missing from .env (intentional)
- > ALLOWED_ORIGINS is a list — supports multiple frontend origins for CORS
- > case_sensitive=True — env var names must match exactly

Verified: `python3 -c "from app.config import get_settings; s=get_settings(); print(s.APP_NAME)"` prints: LogiTwin AI Engine

File 2

app/models.py

Pydantic v2 data models — strict validation layer

Models Defined

Model	Type	Graph Element	Key Validation
SupplierModel	Node	:Supplier	id no spaces, tier must be 1/2/3
FactoryModel	Node	:Factory	capacity_rate 0.0–1.0 float
PortModel	Node	:Port	status enum: operational/congested/closed/mainten
SuppliesRelModel	Relationship	:SUPPLIES	lead_time_days 1–365 int
ShipsThroughRelModel	Relationship	:SHIPS_THROUGH + :DELIVERS_TO	3-10 transit legs in one payload
IngestResponse	Response	All POST endpoints	status, message, node_id
HealthResponse	Response	/health endpoint	status, app, version, database

Verified: All 5 models import and instantiate cleanly with correct property values.

Functions

- > `init_driver()` — called once on startup via FastAPI lifespan. Creates pool of 50 connections. Verifies connectivity before API accepts any request.
- > `close_driver()` — called on shutdown. Drains and closes all connections cleanly. No leaks.
- > `get_driver()` — returns the global driver to any service function that needs to run Cypher.
- > `check_db_health()` — runs RETURN 1 AS ping. Used by /health endpoint. Returns 'connected' or 'unreachable'.

Why a global driver (not per-request)?

Creating a new Neo4j driver per request would be catastrophically slow — each driver creation involves a TCP handshake, authentication, and capability negotiation. A single pooled driver handles all concurrent requests efficiently.

Verified: All 4 functions import cleanly. Driver connects to bolt://localhost:7687 on startup.

Why MERGE not CREATE?

MERGE = 'create if not exists, update if exists'. This makes every ingest operation idempotent — you can send the same payload 100 times and the graph stays consistent with no duplicates.

Functions

Function	Cypher Operation	Returns
<code>upsert_supplier()</code>	MERGE (:Supplier {id}) SET properties	supplier_id string
<code>upsert_factory()</code>	MERGE (:Factory {id}) SET properties	factory_id string
<code>upsert_port()</code>	MERGE (:Port {id}) SET properties	port_id string
<code>upsert_supplies_relationship()</code>	MATCH both nodes, MERGE [:SUPPLIES] relationship	relationship string
<code>upsert_ships_through_relationship()</code>	MATCH 3 nodes, MERGE both [:SHIPS_THROUGH] and [:DELIVERS_TO]	ship_id string
<code>get_all_suppliers()</code>	MATCH (:Supplier) RETURN ordered by tier,name	list[dict]
<code>get_all_factories()</code>	MATCH (:Factory) RETURN ordered by name	list[dict]
<code>get_all_ports()</code>	MATCH (:Port) RETURN ordered by name	list[dict]

Verified: All 8 functions import cleanly. MERGE queries write and read nodes correctly.

All 8 Endpoints

Method	Path	Purpose	Status Code
POST	/ingest/supplier	Upsert a Supplier node	201
GET	/ingest/suppliers	List all Supplier nodes	200
POST	/ingest/factory	Upsert a Factory node	201
GET	/ingest/factories	List all Factory nodes	200
POST	/ingest/port	Upsert a Port node	201
GET	/ingest/ports	List all Port nodes	200
POST	/ingest/relationship/supplies	Create SUPPLIES relationship	201
POST	/ingest/relationship/ships-through	Create full maritime path	201

Data integrity rule: Relationship endpoints require both nodes to exist first. Attempting to create a relationship before ingesting the nodes returns HTTP 422 with a clear error message.

Verified: All 8 routes registered. Routes never write Cypher — they call service functions only.

What main.py does

- > Async lifespan: calls `init_driver()` on startup, `close_driver()` on shutdown
- > CORS middleware: allows frontend origins defined in `.env` `ALLOWED_ORIGINS`
- > Request timing middleware: logs every request with method, path, status, and milliseconds
- > Global exception handler: catches any unhandled error, returns clean JSON — no raw tracebacks
- > GET `/health`: returns healthy only when both API and Neo4j are confirmed live
- > GET `/`: root endpoint returns engine name, version, status, docs URL
- > Mounts `ingest_router` with all 8 endpoints

Verified: App starts, connects to Neo4j, logs startup sequence correctly.

File 7

.env + requirements.txt + docker-compose.yml + Dockerfile

Config, deps, and container definitions

.env — Environment Configuration

```
APP_NAME=LogiTwin AI Engine
APP_VERSION=1.0.0
DEBUG=True
NEO4J_URI=bolt://localhost:7687
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=LogiTwin2024!
API_HOST=0.0.0.0
API_PORT=8000
ALLOWED_ORIGINS=["http://localhost:3000", "http://localhost:8000"]
```

NEVER commit .env to Git. Add it to .gitignore immediately: `echo '.env' >> .gitignore`

Neo4j in Docker — Start Command

```
docker run -d \
--name logitwin-neo4j \
-p 7474:7474 -p 7687:7687 \
-e NEO4J_AUTH=neo4j/LogiTwin2024! \
--restart unless-stopped \
neo4j:5.20.0
```

This is the permanent fix for WSL2 <-> Neo4j Desktop networking. Run Neo4j in Docker, same network as FastAPI.

03 — Live API Test Results

All tests run against the live API at <http://localhost:8000>

Test	Endpoint	Result	Status
Root	GET /	Returns engine name, version, docs URL	PASS
Health	GET /health	status: healthy, database: connected	PASS
Ingest Supplier	POST /ingest/supplier	Tata Components Ltd ingested, node_id returned	PASS
Ingest Port	POST /ingest/port	JNPT Mumbai ingested, node_id returned	PASS
Ingest Factory	POST /ingest/factory	Delhi Assembly Plant ingested	PASS
Guard test	POST /ingest/relationship/supplies	Correctly rejected — factory not yet in graph	PASS
SUPPLIES rel	POST /ingest/relationship/supplies	sup-001-SUPPLIES->fac-001 created	PASS
Maritime path	POST /ingest/relationship/ships-through	sup-001->port-001->fac-001 created	PASS
List suppliers	GET /ingest/suppliers	Returns Tata Components Ltd from graph	PASS

04 — Graph Schema — Live in Neo4j

Node / Relationship	Properties	Status
(:Supplier)	id, name, tier, location	Live
(:Factory)	id, name, capacity_rate, location	Live
(:Port)	id, name, average_clearance_days, status	Live
[SUPPLIES]	component_id, lead_time_days	Live
[SHIPS_THROUGH]	transit_days (supplier to port)	Live
[DELIVERS_TO]	transit_days (port to factory)	Live

05 — What's Next: Subsystem 2 Preview

Subsystem 2 builds the Stochastic Simulation Engine on top of what we built today. It will detect disruptions, traverse the graph to find all affected factories, and score the impact.

- > Celery worker pool wired to Redis broker — both as Docker Compose services
- > Mock disruption event injectors: port congestion, weather strikes, capacity degradation
- > Graph traversal engine: BFS over Neo4j to find all factories downstream of a disrupted node
- > Impact scoring: affected factories ranked by lead_time_days x capacity_rate_loss
- > POST /simulation/run — triggers async Celery worker for a disruption scenario
- > Disruption result written back to Neo4j as :DISRUPTION_EVENT node with severity score

Session saved. To resume: paste your system prompt + say "Resume LogiTwin — Subsystem 1 complete, begin Subsystem 2."